

Tecnologico Superior De Rio Verde

Materia: Programacion Orientada a Objetos

**Práctica: Semana 9 – Sobrecarga de métodos y
operadores**

Alumno: Issac Martin Recendiz Hernandez

Fecha: 15 de Abril del 2026

Introducción

En esta práctica se trabajó el tema de la **sobrecarga de métodos y operadores en C++**, el cual permite que una misma función u operador tenga distintos comportamientos dependiendo de los parámetros que recibe.

Se analizó un programa basado en un personaje tipo Pac-Man, el cual puede moverse de distintas formas y acumular puntos. El objetivo principal fue comprender cómo funciona la sobrecarga y cómo influye en el comportamiento de los objetos dentro de un programa.

Desarrollo

Pacman.h

```
#ifndef PACMAN_H
#define PACMAN_H

#include <iostream>
using namespace std;

class Pacman {
private:
    int x;
    int y;
    int puntos;

public:
    // Constructor
    Pacman();

    // Métodos mover (SOBRECARGA)
    void mover();           // movimiento por defecto
    void mover(char direccion); // mover por dirección
    void mover(int pasos);  // mover por número de pasos

    // Sobrecarga de operadores
    void operator+(int valor); // suma puntos
    void operator+=(int valor); // suma acumulativa

    // Mostrar estado
    void mostrar();
};

#endif
```

Pacman.cpp

```
#include "Pacman.h"
```

```
// Constructor
```

```
Pacman::Pacman() {  
    x = 0;  
    y = 0;  
    puntos = 0;  
}
```

```
// mover() sin parámetros
```

```
void Pacman::mover() {  
    x++;  
    cout << "Pacman se movio a la derecha por defecto\n";  
}
```

```
// mover(char direccion)
```

```
void Pacman::mover(char direccion) {  
    switch(direccion) {  
        case 'w': y++; break; // arriba  
        case 's': y--; break; // abajo  
        case 'd': x++; break; // derecha  
        case 'a': x--; break; // izquierda  
        default:  
            cout << "Direccion invalida\n";  
            return;  
    }  
    cout << "Pacman se movio en direccion: " << direccion << endl;  
}
```

```
// mover(int pasos)
```

```
void Pacman::mover(int pasos) {  
    x += pasos;  
    cout << "Pacman se movio " << pasos << " pasos a la derecha\n";  
}
```

```
// operador +
```

```
void Pacman::operator+(int valor) {  
    puntos += valor;  
    cout << "Se sumaron " << valor << " puntos\n";  
}
```

```
// operador +=
```

```
void Pacman::operator+=(int valor) {  
    puntos += valor;  
    cout << "Se acumularon " << valor << " puntos\n";  
}
```

```
// mostrar estado
```

```

void Pacman::mostrar() {
    cout << "Posicion: (" << x << ", " << y << ")\n";
    cout << "Puntos: " << puntos << endl;
}

```

Main.cpp

```

#include <iostream>
#include "Pacman.h"

using namespace std;

int main() {

    Pacman pacman;

    // Movimiento sin parámetros
    pacman.mover();
    pacman.mostrar();

    cout << "-----\n";

    // Movimiento con dirección
    pacman.mover('w');
    pacman.mostrar();

    cout << "-----\n";

    // Movimiento con pasos
    pacman.mover(3);
    pacman.mostrar();

    cout << "-----\n";

    // Sobrecarga de operadores
    pacman + 10;
    pacman.mostrar();

    cout << "-----\n";

    pacman += 5;
    pacman.mostrar();

    return 0;
}

```

```
1 #ifndef PACMAN_H
2 #define PACMAN_H
3
4 #include <iostream>
5 using namespace std;
6
7 class Pacman {
8 private:
9     int x;
10    int y;
11    int puntos;
12
13 public:
14    // constructor
15    Pacman();
16
17    // Métodos mover (SOBRECARGA)
18    void mover(); // movimiento por defecto
19    void mover(char direccion); // mover por dirección
20    void mover(int pasos); // mover por número de pasos
21
22    // Sobrecarga de operadores
23    void operator+(int valor); // suma puntos
24    void operator+=(int valor); // suma acumulativa
25
26    // Mostrar estado
27    void mostrar();
28
29 };
30 #endif
```

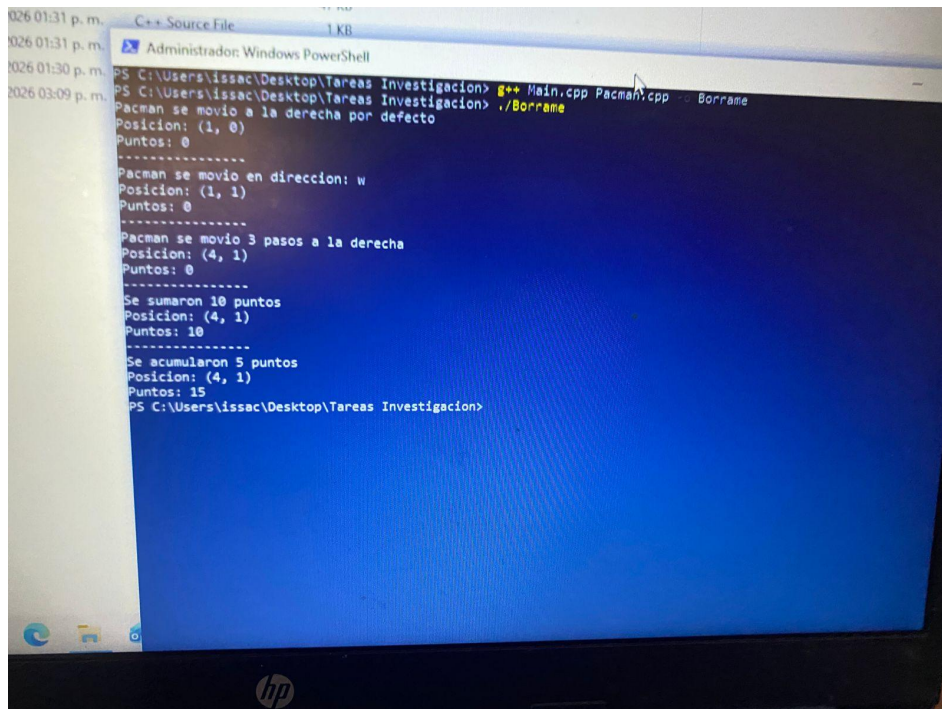
Archivo .h Se muestra la declaración de la clase Pacman, donde se definen los atributos y los métodos. Aquí se observan las diferentes versiones del método agente de mudanzas, lo que indica que hay sobrecarga.

```
11 void Pacman::mover() {
12     x++;
13     cout << "Pacman se movio a la derecha por defecto\n";
14 }
15
16 // mover(char direccion)
17 void Pacman::mover(char direccion) {
18     switch(direccion) {
19         case 'u': y--; break; // arriba
20         case 'd': y++; break; // abajo
21         case 'l': x++; break; // derecha
22         case 'r': x--; break; // izquierda
23         default:
24             cout << "direccion invalida\n";
25             return;
26     }
27     cout << "Pacman se movio en direccion: " << direccion << endl;
28 }
29
30 // mover(int pasos)
31 void Pacman::mover(int pasos) {
32     x += pasos;
33     cout << "Pacman se movio " << pasos << " pasos a la derecha\n";
34 }
35
36 // operador +
37 void Pacman::operator+(int valor) {
38     puntos += valor;
39     cout << "Se sumaron " << valor << " puntos\n";
40 }
41
42 // operador +=
43 void Pacman::operator+=(int valor) {
44     puntos += valor;
45     cout << "Se acumularon " << valor << " puntos\n";
46 }
47
48 // mostrar estado
49 void Pacman::mostrar() {
50     cout << "Posición: " << x << ", " << y << "\n";
51     cout << "Puntos: " << puntos << endl;
52 }
```

Archivo .cpp Se observa la implementación de los métodos. Cada versión de agente de mudanzas realiza una acción distinta dependiendo del parámetro recibido.

```
1 #include <iostream>
2 #include "Pacman.h"
3
4 using namespace std;
5
6 int main() {
7     Pacman pacman;
8
9     // Movimiento sin parámetros
10    pacman.mover();
11    pacman.mostrar();
12
13    cout << "-----\n";
14
15    // Movimiento con dirección
16    pacman.mover('u');
17    pacman.mostrar();
18
19    cout << "-----\n";
20
21    // Movimiento con pasos
22    pacman.mover(5);
23    pacman.mostrar();
24
25    cout << "-----\n";
26
27    // Sobrecarga de operadores
28    pacman + 10;
29    pacman.mostrar();
30
31    cout << "-----\n";
32
33    pacman += 5;
34    pacman.mostrar();
35
36    return 0;
37 }
```

main.cpp Aquí se crea el objeto Pacman y se llaman los métodos. Se puede ver cómo el programa utiliza diferentes versiones de agente de mudanzas.



```
026 01:31 p. m. C++ Source File 1 KB
026 01:31 p. m. Administrador: Windows PowerShell
PS C:\Users\issac\Desktop\Tareas Investigacion> g++ Main.cpp Pacman.cpp -o Borname
026 01:30 p. m.
026 03:09 p. m. PS C:\Users\issac\Desktop\Tareas Investigacion> ./Borname
Pacman se movio a la derecha por defecto
Posicion: (1, 0)
Puntos: 0
-----
Pacman se movio en direccion: w
Posicion: (1, 1)
Puntos: 0
-----
Pacman se movio 3 pasos a la derecha
Posicion: (4, 1)
Puntos: 0
-----
Se sumaron 10 puntos
Posicion: (4, 1)
Puntos: 10
-----
Se acumularon 5 puntos
Posicion: (4, 1)
Puntos: 15
PS C:\Users\issac\Desktop\Tareas Investigacion>
```

Análisis

Sobrecarga de métodos

El programa tiene varias versiones del método agente de mudanzas, por ejemplo:

- agente de mudanzas() → mueve al personaje con un comportamiento por defecto
- mover(char direccion) → mueve según la dirección (arriba, abajo, etc.)
- mover(int pasos) → mueve cierta cantidad de pasos

Esto permite que un mismo método tenga diferentes usos.

¿Cuál se ejecuta en cada caso?

Depende de cómo se llame el método:

- Si no se manda parámetro → se ejecuta agente de mudanzas()
- Si se manda un carácter → se ejecuta mover(carácter)
- Si se manda un entero → se ejecuta mover(int)

¿Cómo decide el programa?

El compilador decide qué método usar basándose en:

- Tipo de dato enviado
- Número de parámetros

Esto se llama **resolución de sobrecarga**.

Sobrecarga de operadores

Operador+

El operador+ fue redefinido para que el objeto pueda **sumar puntos** o modificar su estado.

Ejemplo:

pacman + 10;

Esto aumenta los puntos del personaje.

Operador+=

El operador+= también modifica el objeto, pero de forma acumulativa:

pacman += 5;

Esto suma directamente los puntos al objeto.

Cambios en el objeto

Después de usar los operadores:

- Aumenta el puntaje
- Puede cambiar la posición o estado
- Se reflejan los cambios en pantalla

Flujo de la información

1. El main.cpp llama a los métodos
2. Se envían datos (parámetros)
3. El.cpp ejecuta la lógica
4. El objeto cambia su estado
5. Se muestran los resultados en pantalla

Conclusiones

En esta práctica aprendí que la sobrecarga permite reutilizar funciones con diferentes comportamientos, lo cual hace el código más flexible y organizado.

También comprendí cómo los operadores pueden redefinirse para trabajar con objetos, lo que facilita la manipulación de datos.

Una dificultad fue entender cómo el programa decide qué método usar, pero con el análisis se logró comprender.

Este conocimiento es útil en la vida real para desarrollar programas más eficientes y fáciles de mantener.